DA2304: Introduction to Computer Systems

Assignment 1: Memory Architecture, Hardware Multipliers, and Toolchains

Data Science and AI Department (DSAI)
Indian Institute of Technology Madras

`Instructor:` Dr. Patanjali SLPSK (`patanjali@dsai.iitm.ac.in`)

Release Date: February 26, 2026
Deadline: March 13, 2026, 11:59 PM

Introduction

This assignment merges hardware and software. In **Part 1**, you transition from combinational to sequential logic, building 1-bit memory elements, 16-bit Registers, and ultimately a Random Access Memory (RAM) unit. In **Part 2**, you will extend the standard Hack Architecture by implementing a hardware multiplier, and then build a custom Python Assembler and Jack Compiler to utilize this new hardware feature.

Use the IDE at NAND2Tetris Web IDE to implement and test your hardware designs.

Prerequisites

1. Access to the NAND2Tetris Web IDE.
2. Understanding of Data Flip-Flops (DFF), Multiplexers, and sequential logic timing.
3. Proficiency in Python for writing the Assembler and Compiler logic.
4. Familiarity with the Hack Machine Language and Jack High-Level Language.

Part 1: Registers and Memory Architecture

1 Exercise 1: The 1-Bit Register (Bit) vs. DFF

1.1 Implementation

Design a 1-Bit Register (`Bit`) that can read and write data based on a load signal. You are provided with a primitive Data Flip-Flop (`DFF`) chip.

- Inputs: `in` (1-bit), `load` (1-bit)
- Output: `out` (1-bit)
- Function: If $load = 1$, then $out(t+1) = in(t)$, else $out(t+1) = out(t)$.

1.2 Theoretical Comparison

Compare a simple DFF with the 1-Bit Register you just built. What are the key differences in their behavior regarding state preservation?

1.3 Implementation

Using the 1-bit register implement a 1-bit counter. Functionality of the counter:

- Initial state = 0
- When clock ticks, takes previous state output and increments it by 1

Do you need a Flop or a register for this counter?

2 Exercise 2: The 16-bit Register

2.1 Implementation:

Using the 1-bit `Bit` register from Exercise 1, create a 16-bit Register.

3 Exercise 3: Byte-Level Access (Two 8-bit Registers) RAM2

Instead of a single monolithic 16-bit register, imagine a system that constructs a 16-bit word using two distinct 8-bit registers.

Questions:

1. Can you design this architecture with just the signals from the previous design of a single 16-bit register viz, in, load, and out signals? Yes or No. Justify.
2. If your answer is no to the previous question explain the extra input/output signal that you added to fix the issue?
3. Explain the advantages and disadvantages of read and write operations when you are loading the data 8-bits at a time vs 16-bits at a time.

4 Exercise 4: Folding into RAM8

Now that you can store words, let's design a memory bank of 8 16-bit registers `RAM8` (an 8x16-bit memory).

Questions:

1. Design your RAM8 architecture.
2. Write down the IN and OUT signals used for implementing the RAM8 architecture.
3. How many bits are required to uniquely index each of the registers?

5 Exercise 5: Memory Sizing (1KB)

1. How many individual **bits** are needed to represent 1 Kilobit (Kb)?
2. How many individual **bits** are needed to represent 1 Kilobyte (KB)?
3. How many individual **bits** are needed to represent 16KB?

6 Exercise 6: Scaling Up Memory

How many of the following standard chips would you need to build a 1KB memory?

1. `RAM8` chips?
2. `RAM16` chips?
3. `RAM64` chips?

7 Exercise 7: Address Line Management

1. How are the address lines for the 1KB memory mapped when using RAM8 vs RAM64?
2. Does using a larger underlying block (like `RAM64`) change the *total* number of address bits required by the system?
3. **Implement** a `RAM16K` block using the Hack HDL interface.

8 Extra Credit: Architectural Impact (10 pts)

Answer the following in your theoretical submission:

1. What is the architectural and performance impact of building a massive memory module using larger contiguous blocks (e.g., `RAM4K`) versus many small blocks (`RAM8`)? *(Hint: Think about AI workloads).*

2. From a manufacturing/cost standpoint, which architecture is costlier to design and implement for a 16KB memory: using 1K `RAM8` chips or using 8 `RAM1K` chips? Why?

Part 2: Hardware Multipliers and Custom Toolchains

1 Exercise 1: Extending the Hack ALU (Hardware Multiplier)

The standard Arithmetic Logic Unit (ALU) of the Hack computer is elegantly minimalist. It achieves addition, subtraction, and bitwise operations using just six control bits (`zx, nx, zy, ny, f, no`). However, it lacks a dedicated hardware multiplier. Because of this, multiplying two numbers in standard Hack requires software-level loops (calling the OS function `Math.multiply()`), which takes $O(n)$ time relative to the number of bits.

Your objective is to upgrade the CPU by designing a hardware multiplier that computes products in $O(1)$ time, and embedding it into the ALU structure.

Task Breakdown:

1. **Build the Multiplier (`Multiplier.hdl`):** Construct a chip that takes two 16-bit inputs (`a` and `b`) and outputs a product c. Implement a wallace tree multiplier.

2. **Integrate into the ALU (`ALU.hdl`):** Currently the Hack Instructions uses bits `zx, nx, zy, ny, f, no` to support 18 instructions. Design an instruction for supporting MUL using the same bits. What would be the instruction encoding?

3. **Testing your multiplier**

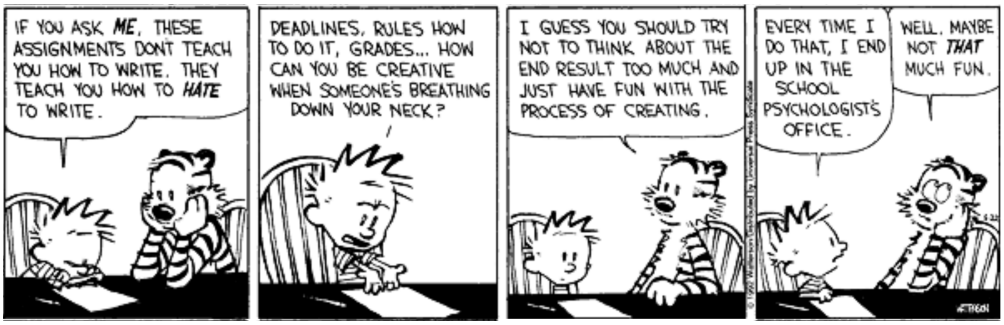
   - Write a test script to validate the multiplier functionality
   - Show that your multiplier functionality does not impact the existing 18 operations of the ALU.

4. **Documentation:** In your PDF report, document the following:

   - Design choice and rationale for the type of adder in each stage of the wallace tree multiplier. Discuss the reduction in addition operations due to the wallace tree strucure.
   - Define your new instruction encoding: Write the value of the 6-bits (`zx, nx, zy, ny, f, no`) that you have chosen to represent the multiplication operation (e.g., `x*y`).
   - If I know that one of my multiplicands is of the format 2^N or $2^N + 1$, then would the multiplier structure change? If yes, discuss the potential changes that would need to be made to the multiplier architecture.

Reference material: Nand2Tetris Project 2: Boolean Arithmetic

Submission Instructions

1. Theoretical answers (Part 1) and your ALU Instruction mappings (Part 2, Ex 1) should be in the report titled `Name_RollNo.pdf`.
2. **Hardware:** Save HDL files as `ChipName.hdl` (e.g., `Register.hdl`, `ALU.hdl`, `Multiplier.hdl`).
3. Compress the PDF, `.hdl` files, and `src/` folder into a **ZIP archive** named: `DA2304_Assignment1_RollNumber.zip`
4. Upload the archive to the course portal by March 13, 2026, 11:59 PM.
5. In the report, add a separate section titled report extra credit, and answer the following question: What is your recent favorite song? (2 points)

Good luck! For queries, post on the course forum or contact the TAs.